

🛠 ИНСТРУКЦИЯ ПО РАСШИРЕННОМУ ИБ-ОТЛАДОЧНОМУ СКАНИРОВАНИЮ БЭКЕНДА (DEBUG / LOGGING)

Настоящее техническое руководство содержит пошаговый регламент конфигурации сквозного логирования, перехвата скрытых крахов Python-рантайма, трассировки асинхронных сокетов Uvicorn и разбора инцидентов безопасности.

КОНТУР СКВОЗНОЙ ОТЛАДКИ И ТРАССИРОВКИ АСИНХРОННЫХ ПОТОКОВ (LOGGING MAP)

Мониторинг аномалий бэкенда FastAPI опирается на трехуровневую фильтрацию событий: Uvicorn-сервер, внутреннее ядро логирования приложения и системный демон `journald` ОС Linux.

--- Схема прохождения пакетов отладки и фильтрации трассировок ---

СЕТЕВОЙ КРАХ ИЛИ АНОМАЛИЯ В `main.py` / `utils.py`

↓ Перехватчик: `logging.getLogger("fastapi")` —> Инициализация логгера

ЭТАП А: ПРИНУДИТЕЛЬНОЕ ПЕРЕКЛЮЧЕНИЕ УРОВНЯ ОЗУ-ФИЛЬТРАЦИИ

`logger.setLevel(logging.DEBUG)` —> Форматирование: `%(asctime)s [%(levelname)s] %(message)s`

↓ Трансляция потоков вывода `sys.stdout` / `sys.stderr`

Контур А: Аварийный крах (`exc_info=True`)

- 🚨 Выброс полного Stack Trace (блока трассировки Python).
- 📄 Выявление точной строки падения со всеми переменными ОЗУ.
- 🔒 Маскирование на фронтенд: HTTP 500 без утечки структуры.

Контур Б: Сетевой Debug-дребезг (Trace)

- 🛡 Логирование сырых JSON-пакетов запросов `fetch`.
- 📡 Фиксация состояния пула соединений `psycopg`.
- 🕒 Контроль времени исполнения DDL на удаленных СУБД.

КОНСОЛИДАЦИЯ СЛУЖБОЙ SYSTEMD —> ВЫЧИТКА КОМАНДОЙ JOURNALCTL -U PGRIGHTS

🔧 Пошаговый алгоритм проведения глубокой отладки (Debugging):

- 1. Перевод бэкенда в расширенный режим DEBUG:** Откройте файл `main.py`, найдите секцию инициализации логгера и принудительно переведите его в режим максимальной детализации. Измените уровень логирования: `logger.setLevel(logging.DEBUG)` При запуске Uvicorn-сервера через терминал или systemd-юнит обязательно пропишите флаг отладки: `uvicorn main:app --host 127.0.0.1 --port 8000 --log-level debug`

2. **Запуск живого мониторинга системного потока:** Откройте SSH-терминал сервера панели и запустите непрерывное чтение логов службы, отсекая старый мусор: `journalctl -u pgrights.service -f -n 50` Сделайте тестовое действие в веб-интерфейсе панели. На экране терминала мгновенно отобразится прохождение HTTP-пакета, включая ID сессии, роль оператора и сгенерированный SQL-запрос.
3. **Локализация скрытых синтаксических ошибок (Traceback):** При возникновении непредвиденных крахов (ошибка 500 в браузере), бэкэнд благодаря конструкции `logger.error(..., exc_info=True)` выведет в журнал полный многострочный Traceback. Найдите верхнюю строчку блока — там будет указан точный файл (например, `main.py`), номер строки кода и тип системного исключения Python (например, `KeyError`, `TypeError`, `ValueError`).

ИСПРАВЛЕННАЯ СЛУЖЕБНАЯ ИНСТРУКЦИЯ ПО ОПЕРАТИВНОМУ ДЕБАЖУ

Контур обеспечивает экспресс-диагностику низкоуровневых сбоев рантайма, пулов сокетов и криптографических ошибок при развертывании приложения.

--- Матрица диагностических кодов и ИБ-затворов бэкэнда ---

СИСТЕМНЫЙ МАРКЕР ОШИБКИ И СТАТУС В ЖУРНАЛЕ КОНТРОЛЯ

 OperationalError: connection to server at "127.0.0.1", port 5432 failed: Connection refused	Причина: Физическое падение службы PostgreSQL-18 или блокировка порта локальным Firewalld.  Фикс: Выполнить <code>systemctl restart postgresql-18</code> и проверить порты утилитойss.
 InvalidToken / Cryptography.fernet.InvalidToken	Причина: Скомпрометирован или изменен MASTER_KEY в файле настроек .env.  Фикс: Вернуть старый ключ расшифровки или принудительно перерегистрировать пароли всех серверов.
 TypeError: SQL identifier parts must be strings, got (tuple) instead	Причина: В класс sql.Identifier передан сырой кортеж fetchone() вместо чистой строки.  Фикс: Распаковать кортеж в коде Python по первому индексу: <code>row[0]</code> .

КОНТУР ОПЕРАТИВНОГО ТЕСТИРОВАНИЯ И ДЕБАЖА УСПЕШНО ЗАКРЫТ

-  Профиль ИБ-безопасности при проведении отладочных работ:
- **Защита от утечки системных отладочных данных (Debug Data Leakage):** При переводе логгера в режим `DEBUG`, детальные трассировки пишутся исключительно во внутренние журналы сервера (systemd journald). На внешний интерфейс клиента в браузер (Network) при любых крахах

по-прежнему отдается только изолированный, абстрактный заголовок `detail: "Ошибка выполнения запроса на удаленном сервере СУБД."`. Это полностью исключает риск проведения разведки структуры кода злоумышленником через интерфейс.

- **Санитизация и очистка журналов:** Логирование паролей в открытом виде внутри `journald` намертво заблокировано маскированием переменных на уровне функций `utils.py` и `main.py`.